

A Minor Project Synopsis
on
Music Recognition Quiz

Submitted to Manipal University Jaipur
towards the partial fulfillment for the award of the degree of

Bachelor of Technology
In
Information Technology

By

Devansh Singh

229302517

Under the Guidance of

Ms. Anju Yadav



MANIPAL UNIVERSITY
JAIPUR

Department of Information Technology

School of CSE & IT

Manipal University Jaipur

2024-2025

Synopsis

1. Introduction

Music is an integral part of human culture and entertainment, with digital platforms revolutionizing how people engage with songs. Music quizzes have emerged as a fun and interactive way to test one's music knowledge. However, most existing platforms rely on third-party APIs, which introduce several challenges such as access restrictions, dependency on external services, and inconsistent availability of song previews.

The Music Recognition Quiz is a web-based application designed to offer an engaging and independent music quiz experience. Unlike traditional platforms, this quiz self-hosts music previews, ensuring unrestricted access and seamless integration. The application features a visually appealing and interactive UI, including animated transitions, a countdown timer, scoring mechanisms, and an immersive quiz flow. Users will listen to short song previews and must correctly identify the song title from a list of multiple-choice options. The project aims to create a competitive and enjoyable platform while addressing the limitations of API-dependent music recognition systems.

2. Problem Statement

Existing music-based quiz applications face several challenges due to reliance on external APIs such as Spotify, YouTube, and JioSaavn. These challenges include:

Limited access to previews due to API restrictions.

Region-based constraints, making some songs unavailable in certain locations.

Unpredictable service availability, leading to disruptions in the user experience.

This project proposes the development of a self-contained music recognition quiz that eliminates dependency on third-party APIs by storing and managing song previews locally. The application will ensure consistent playback, seamless song access, and an uninterrupted gaming experience. Additionally, it will incorporate gamification elements, including scoring, leaderboards, and timed rounds, to enhance user engagement and competitiveness.

3. Motivation

Music quizzes are widely popular among music enthusiasts, but **most existing solutions rely on third-party services, which limit their accessibility and efficiency.** By developing a **self-hosted alternative**, this project offers **faster load times, a more reliable user experience, and complete control over song previews.**

Additionally, the project provides a **hands-on learning opportunity in full-stack web development**, covering **frontend UI design, backend server setup, audio file management, and interactive quiz logic**. The integration of **JavaScript-based game mechanics, visually appealing UI elements, and optimized song storage solutions** makes this quiz both technically challenging and user-friendly.

4. Objective:

The primary objectives of the Music Recognition Quiz are:

1. **To develop an engaging music quiz platform** with an interactive user interface.
2. **To implement a self-hosted music preview system**, reducing reliance on third-party APIs.
3. **To create a gamified experience with multiple rounds, scoring, and a countdown timer.**
4. **To ensure smooth song selection and playback**, optimizing user experience and system performance.

5. Methodology/ Planning of work:

The project is structured into six major phases, each focusing on different aspects of development:

1. Requirement Analysis & Research – Identifying quiz requirements, studying existing platforms, and finalizing the game design.
2. Frontend Development – Designing and implementing UI components with animations, responsive design, and a game-friendly interface.
3. Backend Development – Developing a Node.js server to serve self-hosted music previews and manage game logic.
4. Game Logic Implementation – Adding the scoring system, countdown timer, multiple-choice selection, and round progression.
5. Testing & Debugging – Performing UI/UX testing, fixing bugs, and optimizing the gameplay experience.
6. Deployment & Documentation – Hosting the application and preparing final reports.

Phase	Tasks	Duration	Timeline
1. Research & Planning	- Identify quiz requirements - Study existing platforms - Finalize game design	2 weeks	Week 1–2 (1 Feb–15 Feb)
2. UI/UX Development	- Design user interface - Implement animations - Optimize for responsiveness	2 weeks	Week 3–4 (16 Feb–2 Mar)
3. Backend Development	- Set up Node.js server - Implement music storage and retrieval - Develop API endpoints	4 weeks	Week 5–8 (3 Mar–31 Mar)
4. Game Logic Integration	- Implement scoring system - Add countdown timer - Handle answer validation	2 weeks	Week 9–10 (1 Apr–15 Apr)
5. Testing & Debugging	- Perform UI/UX testing - Fix gameplay bugs - Optimize performance	2 weeks	Week 11–12 (16 Apr–30 Apr)
6. Deployment & Documentation	- Deploy final version - Write reports and finalize project presentation	2 weeks	Week 13–14 (1 May–15 May)

5. Facilities required for proposed work:

Software Requirements:

- Frontend Technologies: HTML, CSS, JavaScript (for UI and game logic).
- Backend Technologies: Node.js & Express.js (for serving music previews).
- Development Tools: Visual Studio Code (VS Code) or any text editor.
- Hosting Environment: Local server or cloud-based hosting for deployment.

Hardware Requirements:

- Personal Computer / Laptop (for development and testing).
- Internet Connection (for research and online resources).
- Audio Output Device (Speakers/Headphones for testing music playback).

Bibliography/References:

Mozilla Developer Network (MDN) – Used for JavaScript, HTML, CSS, and Web APIs.
(<https://developer.mozilla.org/>)

Node.js & Express.js Official Documentation – For backend development and handling API requests. (<https://nodejs.org/>)

Spotify Developer API – Researched for fetching song metadata and preview links.
(<https://developer.spotify.com/>)

YouTube Data API – Used to fetch song metadata before switching to a self-hosted approach.
(<https://developers.google.com/youtube/v3>)